॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Certificate

Department of Computer Science and Engineering

This is to certify that this Practical Record contains the bona fide practical work carried out by **Mr./Ms.** _____, bearing **PRN.** _____, of **Batch** _____, **Semester** _____, **Section** _____, during the **Academic Year** _____.

Date: _____

Name of Course Teacher: _____

Signature of Course Teacher: _____

Name of Student: _____ **PRN:** _____

Batch: _____ **Semester:** _____ **Section:** _____

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Index

Practical No.	Aim	Performance Date	Submission Date	Remarks
1	Demonstrate inline, internal, and external JavaScript; use console methods; create a webpage showing user info and a welcome message.			
2	Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.			
3	Implement control structures and form validation; create a grading system based on user-entered marks.			
4	Use function types, scope, and closures; apply trycatch; build a palindrome checker.			
5	Apply array methods and object handling; create a cart total calculator with discount logic.			
6	Use string functions and regex for email validation, data extraction, and text analysis.			
7	Perform DOM traversal and updates; develop a todo list app with add, edit, and delete features.			
8	Access and validate form fields using events; design a gym admission form with live input checks.			
9	Implement localStorage and sessionStorage to save user preferences like theme settings.			
10	Load and display JSON data using fetch() and jQuery $.getJSON(); render results dynamically in a table.			
VA 11	Use JavaScript timers (setTimeout(), setInterval()) and ES6 modules (import/export) to build a dynamic countdown or class scheduler with realtime updates and responsive layout.			

VA 12	Build an interactive real-time web app using JavaScript and jQuery with DOM, events, storage, and AJAX integration.			

Experiment No. 1

AIM/Title

To demonstrate the use of Inline, Internal, and External JavaScript and use console methods such as console.log(), console.error(), console.table(), and console.trace(). Also, create a simple webpage that displays a welcome message and logs user environment information in the browser console.

Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Microsoft Edge
- HTML5
- JavaScript (ES6)

Theory

JavaScript is a client-side scripting language used to make web pages interactive and dynamic. It can be included in an HTML document in three different ways:

Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>JavaScript Experiment 1</title>

  <!-- Internal JavaScript -->
  <script>
    function internalJS() {
```

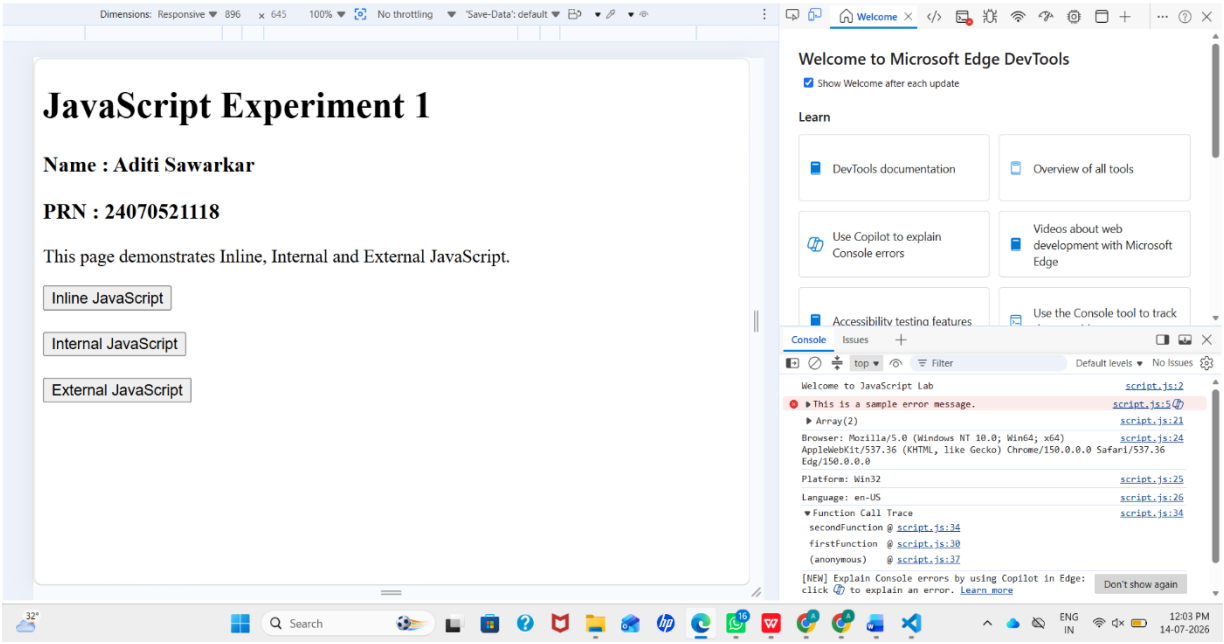
```
        alert("Welcome to JavaScript Lab!");
      }
  </script>
</head>
<body>
  <h1>JavaScript Experiment 1</h1>
  <h3>Name : Aditi Sawarkar</h3>
  <h3>PRN : Your PRN Number</h3>
  <p>
    This page demonstrates Inline, Internal and External JavaScript.
  </p>
  <!-- Inline JavaScript -->
  <button onclick="alert('This is Inline JavaScript')">
    Inline JavaScript
  </button>
  <br><br>
  <!-- Internal JavaScript -->
  <button onclick="internalJS()">
    Internal JavaScript
  </button>
  <br><br>
  <!-- External JavaScript -->
  <button onclick="externalJS()">
    External JavaScript
  </button>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
// Welcome Message
console.log("Welcome to JavaScript Lab");
// Error Message
console.error("This is a sample error message.");
// Table
const students = [
  {
```

```
    Name: "Aditi",
    Branch: "AIML",
    Year: "Third Year"
  },
  {
    Name: "Rahul",
    Branch: "CSE",
    Year: "Second Year"
  }
];
console.table(students);
// User Environment Information
console.log("Browser:", navigator.userAgent);
console.log("Platform:", navigator.platform);
console.log("Language:", navigator.language);
// Trace Example
function firstFunction() {
  secondFunction();
}
function secondFunction() {
  console.trace("Function Call Trace");
}
firstFunction();
// External JavaScript Function
function externalJS() {
  alert("This is External JavaScript.");
}
```

Output Screenshot



Result/Conclusion

Thus, the program was successfully executed to demonstrate Inline, Internal, and External JavaScript. The console methods console.log(), console.error(), console.table(), and console.trace() were used successfully. The webpage displayed a welcome message, and the user's browser environment information was logged in the browser console.

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Experiment No. 2

AIM/Title

To demonstrate the use of var, let, const, template literals, and object destructuring in JavaScript by developing a simple Billing Calculator using user input.

Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Microsoft Edge
- HTML5
- JavaScript (ES6)

Theory

JavaScript provides three ways to declare variables:

- var: Function-scoped and can be redeclared.
- let: Block-scoped and can be updated but not redeclared in the same scope.
- const: Block-scoped and cannot be reassigned after initialization.

Template Literals use backticks (`) and ${} to embed variables in strings.
Destructuring is an ES6 feature that extracts values from objects or arrays into separate variables.
In this experiment, a Billing Calculator accepts the customer's name, product name, quantity, and price. It calculates the total bill and displays the result using template literals. It also demonstrates object destructuring.

Code

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Billing Calculator</title>
</head>
<body>
<h1>JavaScript Experiment 2</h1>
<h3>Name : Aditi Sawarkar</h3>
<h3>PRN : 24070521118</h3>
```

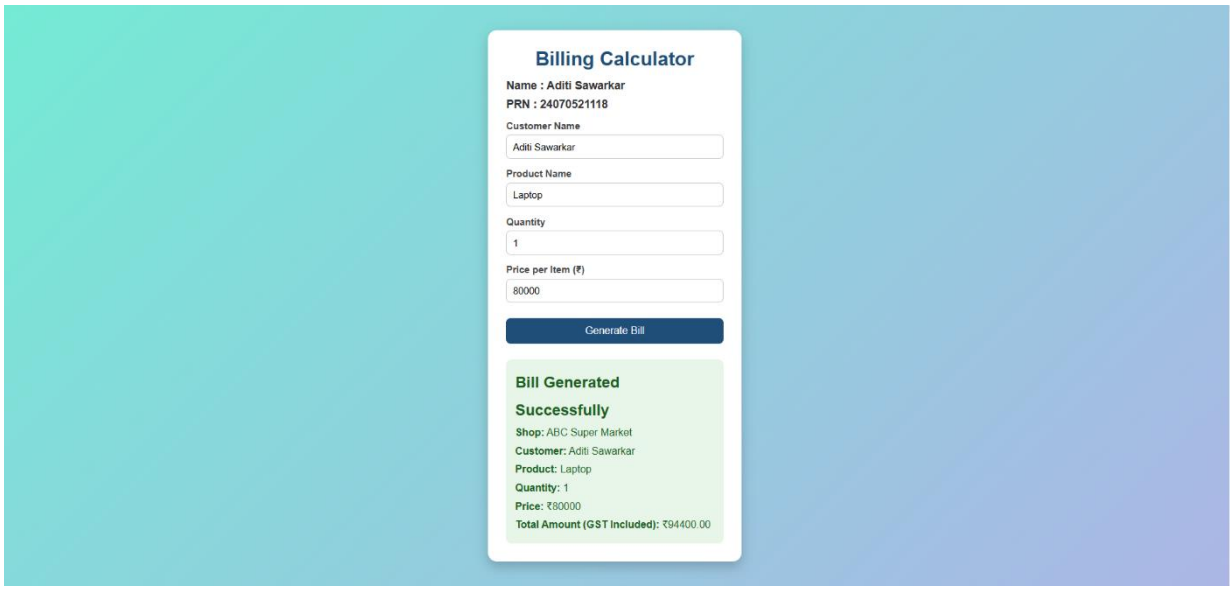
```
<h2>Billing Calculator</h2>
<label>Customer Name:</label><br>
<input type="text" id="customer"><br><br>
<label>Product Name:</label><br>
<input type="text" id="product"><br><br>
<label>Quantity:</label><br>
<input type="number" id="quantity"><br><br>
<label>Price per Item:</label><br>
<input type="number" id="price"><br><br>
<button onclick="calculateBill()">Generate Bill</button>
<h2 id="result"></h2>
<script src="script.js"></script>
</body>
</html>
```

script.js

```
// var declaration
var shopName = "ABC Super Market";
// const declaration
const gst = 0.18;
// Function
function calculateBill() {
   // let declaration
   let customer = document.getElementById("customer").value;
   let product = document.getElementById("product").value;
   let quantity = Number(document.getElementById("quantity").value);
   let price = Number(document.getElementById("price").value);
   let total = quantity * price;
   let gstAmount = total * gst;
   let finalAmount = total + gstAmount;
   // Object
   const bill = {
      customer,
      product,
      quantity,
      price,
      finalAmount
```

```
  };
  // Object Destructuring
  const { customer: cname, product: pname, finalAmount: amount } = bill;
  // Template Literal
  document.getElementById("result").innerHTML = `
    <h3>Bill Generated Successfully</h3>
    <p><b>Shop:</b> ${shopName}</p>
    <p><b>Customer:</b> ${cname}</p>
    <p><b>Product:</b> ${pname}</p>
    <p><b>Total Amount (Including GST): ₹${amount.toFixed(2)}</p>
  `;
console.log("Bill Details:", bill);
}
```

Output Screenshot



Result/Conclusion

Thus, the program was successfully executed to demonstrate the use of var, let, const, template literals, and object destructuring. A Billing Calculator was developed that accepts user input, calculates the total amount including GST, and displays the bill successfully.

Experiment No. 3

Title

To implement JavaScript control structures and form validation by developing a Student Grading System based on user-entered marks.

Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Microsoft Edge
- HTML5
- CSS3
- JavaScript (ES6)

Theory

JavaScript control structures are used to make decisions based on conditions. The if-else statement checks whether a condition is true or false and executes the appropriate block of code.

Form validation ensures that users enter valid data before processing it. In this experiment, marks entered by the user are validated to ensure they are between 0 and 100. Based on the entered marks, the program calculates the student's grade using if-else conditions.

Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JavaScript Experiment 3</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h1>Student Grading System</h1>
<h3>Name : Aditi Sawarkar</h3>
<h3>PRN : 24070521118</h3>
<label>Student Name</label>
<input type="text" id="name" placeholder="Enter Student Name">
<label>Enter Marks (0-100)</label>
```

```
<input type="number" id="marks" placeholder="Enter Marks">
<button onclick="calculateGrade()">
Check Grade
</button>
<div id="result"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

style.css

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial, sans-serif;
}
body{
background:linear-gradient(135deg,#6dd5fa,#2980b9);
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}
.container{
background:white;
padding:30px;
width:420px;
border-radius:15px;
box-shadow:0 10px 25px rgba(0,0,0,0.3);
}
h1{
text-align:center;
color:#0b5394;
margin-bottom:20px;
}
h3{
```

```
margin-bottom:8px;
color:#333;
}
label{
display:block;
margin-top:15px;
font-weight:bold;
}
input{

width:100%;
padding:10px;
margin-top:6px;
border-radius:8px;
border:1px solid gray;
font-size:16px;
}
button{
margin-top:20px;
width:100%;
padding:12px;
background:#0b5394;
color:white;
border:none;
border-radius:8px;
font-size:18px;
cursor:pointer;
}
button:hover{
background:#073763;
}
#result{
margin-top:20px;
padding:15px;
background:#e8f5e9;
border-radius:10px;
font-size:18px;
```

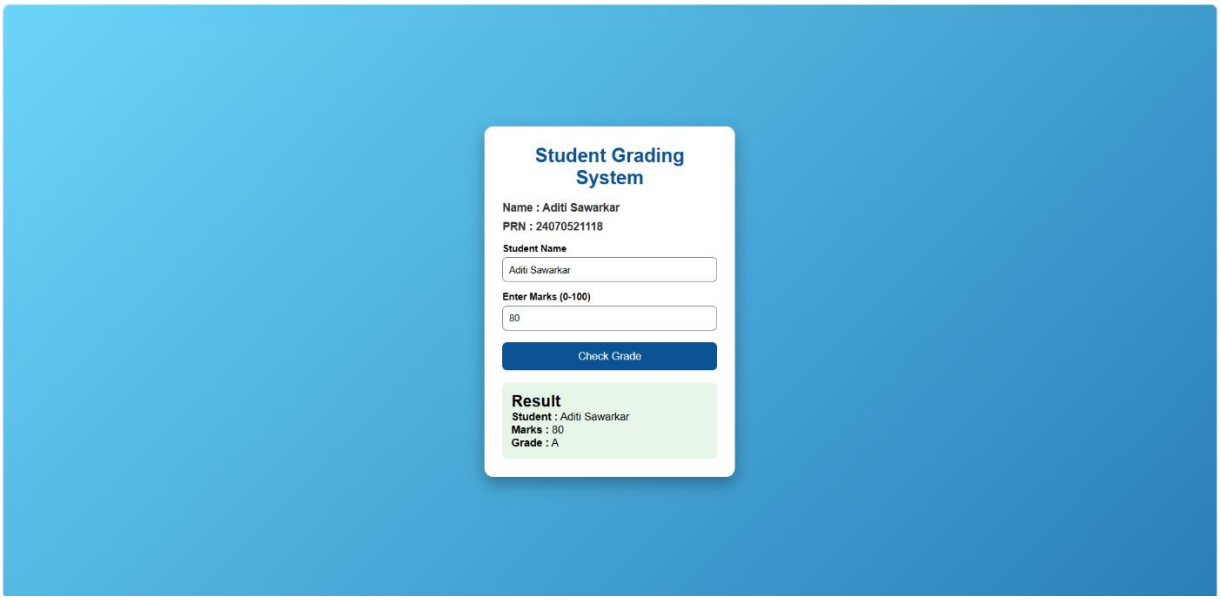
```
}
```

script.js

```
function calculateGrade(){
let name=document.getElementById("name").value;
let marks=Number(document.getElementById("marks").value);
if(name==""){
alert("Please Enter Student Name");
return;
}
if(marks<0 || marks>100 || isNaN(marks)){
alert("Marks should be between 0 and 100");
return;
}
let grade;
if(marks>=90){
grade="A+";
}
else if(marks>=80){
grade="A";
}
else if(marks>=70){
grade="B";
}
else if(marks>=60){
grade="C";
}
else if(marks>=50){
grade="D";
}
else{
grade="F";
}
document.getElementById("result").innerHTML=`
<h2>Result</h2>
<p><b>Student :</b> ${name}</p>
```

```
<p><b>Marks :</b> ${marks}</p>
<p><b>Grade :</b> ${grade}</p>
`;}
```

Output Screenshot



Result/Conclusion

Thus, the program was successfully executed to demonstrate JavaScript control structures (if-else) and form validation. The grading system accepted valid user input, validated the entered marks, and displayed the appropriate grade based on the student's marks.

Experiment No. 4

Title

To demonstrate JavaScript function types, scope, closures, and exception handling using try...catch by developing a Palindrome Checker.

Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Microsoft Edge
- HTML5
- CSS3
- JavaScript (ES6)

Theory

JavaScript functions are reusable blocks of code that perform specific tasks. Functions can be declared in different ways, such as function declarations, function expressions, and arrow functions.
Scope determines where variables are accessible. Variables declared inside a function are local to that function.
A closure is created when an inner function remembers and accesses variables from its outer function even after the outer function has finished executing.
The try-catch statement is used for exception handling. It allows the program to catch and handle runtime errors without crashing.
In this experiment, the program checks whether a given word is a palindrome (reads the same forward and backward), while demonstrating function types, scope, closures, and exception handling.

Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Palindrome Checker</title>
<link rel="stylesheet" href="style.css">
</head>
```

```
<body>
<div class="container">
<h1>Palindrome Checker</h1>
<h3>Name : Aditi Sawarkar</h3>
<h3>PRN : 24070521118</h3>
<label>Enter a Word</label>
<input type="text" id="word" placeholder="Enter text">
<button onclick="checkPalindrome()">
Check Palindrome
</button>
<div id="result"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

style.css

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial,sans-serif;
}
body{
background:linear-gradient(135deg,#ff9a9e,#fad0c4);
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}
.container{
width:420px;
background:white;
padding:30px;
border-radius:15px;
box-shadow:0 10px 20px rgba(0,0,0,0.2);
}
```

```
h1{
text-align:center;
color:#c2185b;
margin-bottom:20px;
}
h3{
margin-bottom:8px;
}
label{
display:block;
margin-top:15px;
font-weight:bold;
}
input{
width:100%;
padding:10px;
margin-top:8px;
border-radius:8px;
border:1px solid #ccc;
font-size:16px;
}
button{
width:100%;
padding:12px;
margin-top:20px;
background:#c2185b;
color:white;
border:none;
border-radius:8px;
font-size:17px;
cursor:pointer;
}
button:hover{
background:#880e4f;
}
#result{
margin-top:20px;
```

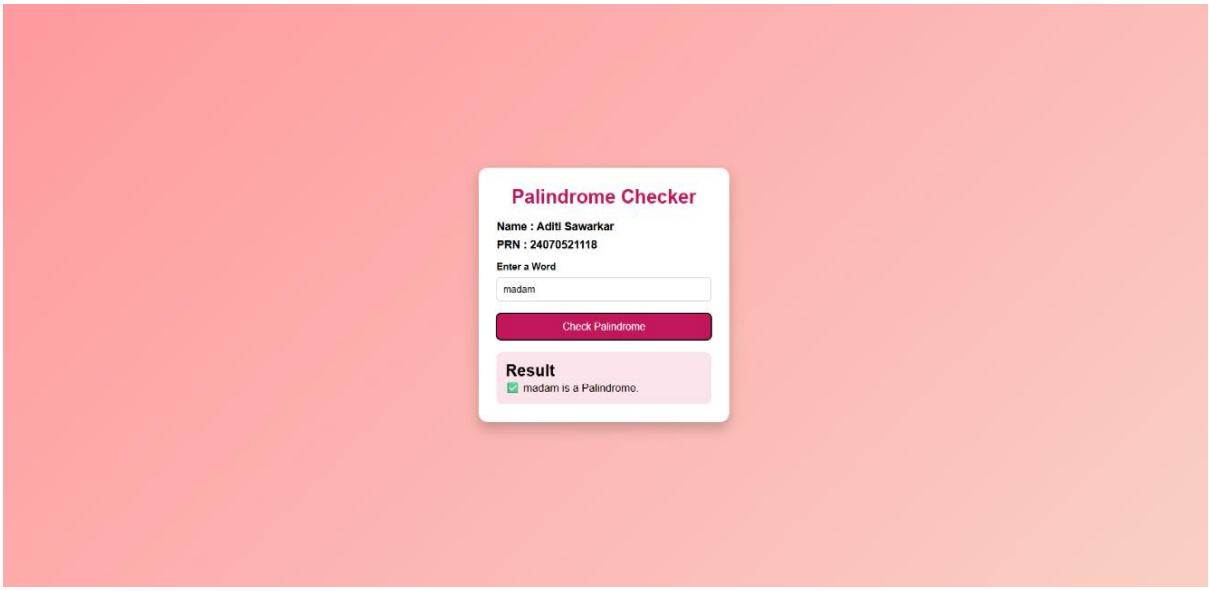
```
padding:15px;
background:#fce4ec;
border-radius:10px;
font-size:18px;
}
```

script.js

```
// Function Declaration
function reverseString(str) {
  return str.split("").reverse().join("");
}
// Closure
function message() {
  let title = "Palindrome Checker";
  return function () {
    console.log(title + " Started");
  };
}
const start = message();
start();
// Arrow Function
const isPalindrome = (text) => {
  return text === reverseString(text);
};
// Main Function
function checkPalindrome() {
  try {
    let word = document.getElementById("word").value.trim().toLowerCase();
    if (word === "") {
      throw "Please enter a word.";
    }
    let result = "";
    if (isPalindrome(word)) {
      result = "✅ " + word + " is a Palindrome.";
    } else {
      result = "❌ " + word + " is NOT a Palindrome.";
    }
```

```
    document.getElementById("result").innerHTML = `
      <h2>Result</h2>
      <p>${result}</p>
    `;
  }
  catch(error) {
    alert(error);
  }
}
```

Output Screenshot



Result/Conclusion

Thus, the program was successfully executed to demonstrate JavaScript function declaration, arrow functions, scope, closures, and exception handling using try...catch. The application correctly identified whether the entered word was a palindrome and handled invalid input gracefully.